FUNCTION OVERLOADING, INLINE FUNCTIONS & DEFAULT ARGUMENTS

ALVINA ANJUM



PREVIOUS LECTURE

- Function Call by Value
- Function Call by reference
- Function Call by Pointers

LECTURE OUTLINE

- Default Arguments
- Function Overloading
- Inline Functions

DEFAULT ARGUMENTS

- *You can define default values for parameters in a function.*
- C++ allows you to declare functions with default argument values. The default values are passed to the parameters when a function is invoked without the arguments.

```
#include <iostream>
using namespace std;

// Display area of a circle
void printArea(double radius = 1)
{
  double area = radius * radius * 3.14159;
  cout << "area is " << area << endl;
}

int main()
{
  printArea();
  printArea(4);

  return 0;
}
```

```
area is 3.14159
area is 50.2654
```

DEFAULT ARGUMENTS

- When a function contains a mixture of parameters with and without default values, those with default values must be declared last. For example, the following declarations are illegal:
  - void t1(int x, int y = 0, int z); // Illegal
  - void t2(int x = 0, int y = 0, int z); // Illegal
  - However, the following declarations are fine:
  - void t3(int x, int y = 0, int z = 0); // Legal
  - void t4(int x = 0, int y = 0, int z = 0); // Legal
- When an argument is left out of a function, all arguments that come after it must be left out as well. For example, the following calls are illegal:

  t3(1, , 20);

  t4(, , 20);

  but the following calls are fine:

  t3(1); // Parameters y and z are assigned a default value

  t4(1,2); // Parameter z is assigned a default value

ACTIVITY

- Which of the following function declarations are illegal?

```
void t1(int x, int y = 0, int z);
void t2(int x = 0, int y = 0, int z);
void t3(int x, int y = 0, int z = 0);
void t4(int x = 0, int y = 0, int z = 0);
```

FUNCTION OVERLOADING

- *Overloading functions enables you to define the functions with the same name as long as their signatures are different.*
- Overloading functions can make programs clearer and more readable. Functions that perform the same task with different types of parameters should be given the same name.
- Overloaded functions must have different parameter lists. You cannot overload functions based on different return types.

```
 1  #include <iostream>
 2  using namespace std;
 3
 4  // Return the max between two int values
 5  int max(int num1, int num2)
 6  {
 7    if (num1 > num2)
 8      return num1;
 9    else
10      return num2;
11  }
12
13  // Find the max between two double values
14  double max(double num1, double num2)
15  {
16    if (num1 > num2)
17      return num1;
18    else
19      return num2;
20  }
21
22  // Return the max among three double values
23  double max(double num1, double num2, double num3)
24  {
25    return max(max(num1, num2), num3);
26  }
27
```

```
28  int main()
29  {
30    // Invoke the max function with int parameters
31    cout << "The maximum between 3 and 4 is " << max(3, 4) << endl;
32
33    // Invoke the max function with the double parameters
34    cout << "The maximum between 3.0 and 5.4 is "
35      << max(3.0, 5.4) << endl;
36
37    // Invoke the max function with three double parameters
38    cout << "The maximum between 3.0, 5.4, and 10.14 is "
39      << max(3.0, 5.4, 10.14) << endl;
40
41    return 0;
42  }
```

AMBIGUOUS INVOCATION

- Sometimes there are two or more possible matches for an invocation of a function, and the compiler cannot determine the most specific match. This is referred to as *ambiguous invocation*.
- Ambiguous invocation causes a compile error. Consider the following code:

AMBIGUOUS INVOCATION

```
#include <iostream>
using namespace std;

int maxNumber(int num1, double num2)
{
  if (num1 > num2)
    return num1;
  else
    return num2;
}

double maxNumber(double num1, int num2)
{
  if (num1 > num2)
    return num1;
  else
    return num2;
}

int main()
{
  cout << maxNumber(1, 2) << endl;

  return 0;
}
```

ACTIVITY

What is wrong in the following program?

```
void p(int i)
{
  cout << i << endl;
}

int p(int j)
{
  cout << j << endl;
}
```

Given two function definitions,

```
double m(double x, double y)
double m(int x, double y)
```

answer the following questions:

a. Which of the two functions is invoked for
```
double z = m(4, 5);
```
b. Which of the two functions is invoked for
```
double z = m(4, 5.4);
```
c. Which of the two functions is invoked for
```
double z = m(4.5, 5.4);
```

INLINE FUNCTIONS

- *C++ provides inline functions for improving performance for short functions.*
- Implementing a program using functions makes the program easy to read and easy to maintain, but function calls involve runtime overhead (i.e., pushing arguments and CPU registers into the stack and transferring control to and from a function).
- C++ provides *inline functions* to avoid function calls. Inline functions are not called; rather, the compiler copies the function code *in line* at the point of each invocation. To specify an inline function, precede the function declaration with the inline keyword,

INLINE FUNCTIONS

inline function

invoke inline function

invoke inline function

```
1  #include <iostream>
2  using namespace std;
3
4  inline void f(int month, int year)
5  {
6    cout << "month is " << month << endl;
7    cout << "year is " << year << endl;
8  }
9
10 int main()
11 {
12   int month = 10, year = 2008;
13   f(month, year);  // Invoke inline function
14   f(9, 2010); // Invoke inline function
15
16   return 0;
17 }
```

```
month is 10
year is 2008
month is 9
year is 2010
```

INLINE FUNCTIONS

- As far as programming is concerned, inline functions are the same as regular functions, except they are preceded with the inline keyword.
- However, behind the scenes, the C++ compiler expands the inline function call by copying the inline function code.
- Inline functions are desirable for short functions but not for long ones that are called in multiple places in a program,
- Making multiple copies will dramatically increase the executable code size. For this reason, C++ allows the compilers to ignore the inline keyword if the function is too long.
- The inline keyword is merely a request; it is up to the compiler to decide whether to honor or ignore it.

NEXT LECTURE

Recursion